Design and Analysis of Algorithms
(E1ITB303)

Micro Project Report

AI-Based Timetable Generation System aligned with NEP 2020 for Multidisciplinary Education Structures

On

Submitted by

Shreya.D	2411021061105
Priyanka.R	2411021060251
Chinthala Hasini	2411021061159

Table of Contents

Abstract

1. **Introduction**
   1.1 Project Overview
   1.2 Objectives of the Project
   1.3 Problem Statement
   1.4 Scope of the Project
2. **Algorithm Design**
   2.1 Description of the Algorithm
   2.2 Flowchart
   2.3 Pseudocode
3. **Complexity Analysis**
   3.1 Time Complexity (Best, Average, Worst Case)
   3.2 Space Complexity
4. **Implementation**
   4.1 Programming Language Used
   4.2 Code Explanation
5. **Results and Discussion**
   5.1 Output
   5.2 Performance Analysis
6. **Conclusion**
7. **Future Scope**

References

Abstract

Timetable generation is one of the most complex and important tasks in educational institutions. With the introduction of multidisciplinary education under NEP 2020, students can choose courses across different domains, which increases the complexity of scheduling. Manual timetable creation often leads to conflicts such as overlapping classes, improper classroom allocation, and instructor clashes.

This project proposes an AI-Based Timetable Generation System that automatically generates a conflict-free timetable using algorithmic techniques from Design and Analysis of Algorithms. The system models the timetable problem as a graph where each course is represented as a vertex and conflicts between courses are represented as edges.

To solve this problem, graph coloring and backtracking algorithms are used. The graph coloring algorithm assigns time slots (colors) to courses such that no two conflicting courses share the same time slot. Backtracking ensures that if a conflict arises, the system reassigns time slots to achieve an optimal solution.

The system ensures that:

No instructor is assigned multiple courses at the same time

No classroom is used for more than one course simultaneously

All courses are scheduled within available time slots

This approach guarantees correctness while efficiently handling constraints. The system is scalable and can be extended for large institutions. It demonstrates the application of NP-hard problem solving techniques in real-world scenarios.

1.Introduction

1.1 Project Overview

In modern educational systems, especially after NEP 2020, students are allowed to select courses from multiple disciplines. This flexibility increases the complexity of timetable generation. Institutions must ensure proper allocation of instructors, classrooms, and time slots without any conflicts.

Traditional manual scheduling is:

Time-consuming

Error-prone

Difficult to manage for large datasets

This project introduces an automated system that uses graph-based algorithms to generate a conflict-free timetable efficiently.

1.2 Objectives of the Project

The main objectives are:

To design an automated timetable generation system

To eliminate scheduling conflicts

To apply graph coloring techniques

To use backtracking for optimal solutions

To efficiently allocate classrooms and instructors

To support NEP 2020 multidisciplinary structure

To analyze time and space complexity

To improve scheduling efficiency

1.3 Problem Statement

The problem is to generate a timetable such that:

Each course is assigned a time slot and classroom

No instructor is assigned more than one course at the same time

No classroom is assigned more than one course at the same time

All courses are scheduled without conflicts

Graph Model

Vertices → Courses

Edges → Conflicts (same instructor / same students / same classroom requirement)

Colors → Time slots

1.4 Scope of the Project

Applicable in schools, colleges, universities

Supports multidisciplinary systems

Can handle large-scale scheduling

Can be integrated with AI tools

Can be extended to real-time scheduling

Useful for automated academic systems

2. Algorithm Design

2.1 Description of the Algorithm

The timetable generation problem is modeled as a graph coloring problem, which is a well-known NP-hard problem.

Concept Used
Graph Representation
Graph Coloring
Backtracking
Working Principle
Represent each course as a vertex
Add edges between courses with conflicts
Assign time slots using colors
Ensure no adjacent vertices have the same color
Use backtracking if conflict occurs
Example

If Course A and Course B have the same instructor → edge exists
They cannot be assigned the same time slot

2.2 Flowchart

Start
↓
Input courses, instructors, classrooms
↓
Create graph
↓
Assign time slot (color)
↓
Check constraints
↓
If valid → continue
Else → backtrack
↓
Repeat
↓
Generate timetable
↓
End

2.3 Pseudocode

```
function isSafe(v, graph, color, c):
   for each node i:
```

```
            if graph[v][i] == 1 and color[i] == c:
                return false
        return true

    function graphColoring(graph, m, color, v):
        if v == number of vertices:
            return true

        for c from 1 to m:
            if isSafe(v, graph, color, c):
                color[v] = c

                if graphColoring(graph, m, color, v+1):
                    return true

                color[v] = 0   // Backtrack

  return false
```

3.Complexity Analysis

3.1 Time Complexity

Graph coloring is an NP-Hard problem.

Worst Case:

$O(k^n)$

Where:

n = number of courses

k = number of time slots

Backtracking explores all possible assignments in worst case.

3.2 Space Complexity

O(n + e)

Where:

n = vertices

e = edges

Used for:

Graph storage

Color array

Recursion stack

4.Implementation

4.1 Programming Language Used

Python / Java

4.2 Code Explanation

Key Components

Graph Representation

Adjacency matrix or list
graph[i][j] = 1 indicates conflict

Color Array

Stores assigned time slots

isSafe Function

Checks conflict condition

Backtracking Function

Assigns and reassigns slots
Sample Code (Python)
def is_safe(v, graph, color, c):

```
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True

def solve(graph, m, color, v):
    if v == len(graph):
        return True

    for c in range(1, m+1):
        if is_safe(v, graph, color, c):
            color[v] = c

            if solve(graph, m, color, v+1):
                return True

            color[v] = 0

    return False

graph = [
    [0,1,1],
    [1,0,1],
    [1,1,0]
]

m = 3
color = [0]*len(graph)
```

```
        if solve(graph, m, color, 0):
            print("Timetable:", color)
        else:
    print("No solution")
```

5.Results and Discussion

5.1 Output

Each course assigned a time slot
No instructor conflict
No classroom conflict
Complete timetable generated

5.2 Performance Analysis

Efficient for small inputs
Backtracking ensures correctness
Performance reduces for large inputs
Can be optimized using heuristics
.

6.Conclusion

The AI-Based Timetable Generation System successfully solves the scheduling problem using graph coloring and backtracking. It ensures conflict-free scheduling and supports modern educational requirements under NEP 2020.

The project highlights the importance of DAA concepts such as graph theory, recursion, and NP-hard problem solving in real-world applications..

7.Future Scope

Use AI/ML for prediction

Genetic algorithms for optimization

Real-time timetable updates

Integration with mobile apps

Cloud-based scheduling systems

8.References

Cormen et al., Introduction to Algorithms
GeeksforGeeks – Graph Coloring
NPTEL DAA Course
NEP 2020 Policy Document
Tutorialspoint